ALPE FORMACIÓN · SANTANDER

Edición 2026 · PHP 8.x

CURSO DE PHP · CLASE 38

Funcionalidades avanzadas

- Eventos y listeners
- Correo electrónico
- Colas y jobs
- Notificaciones
- Caché
- Tareas programadas
- Registros (logs)
- Excepciones propias
- El recorrido completo de una matrícula



Ramón Abramo

+25 años de experiencia docente · Santander, Cantabria

ALPE

FORMACIÓN PROFESIONAL

🧭 Contenido de la clase

Cuando matriculamos a un alumno, en una academia de verdad pasan muchas cosas: se le envía un correo de confirmación, si el curso se llena se avisa a la dirección, la víspera del inicio se le recuerda que empieza... Si metiéramos todo eso en el controlador, tendríamos un método de cien líneas que, además, haría esperar al usuario mientras se envían los correos.

Hoy vemos las herramientas que usa una aplicación profesional para todo eso: **eventos**, **colas**, **correo**, **notificaciones**, **caché**, **tareas programadas** y **registros**. Son muchas piezas, pero todas siguen la misma idea que llevamos repitiendo desde la clase 22: **cada clase, una sola tarea**.

•
👌 Cómo vamos a trabajar



1
📘 Eventos y listeners



2
👍 Correo electrónico



3
😮 Colas y jobs



4
🚀 Notificaciones



5
🔍 Caché



6
🤔 Tareas programadas



7
📘 Registros (logs)



8
⚠️ Excepciones propias



9
🚀 El recorrido completo de una matrícula



10
🚀 Ejercicios



11
👋 Lo que tenéis que llevaros de hoy





CLASE 38

👌 Cómo vamos a trabajar

Copiad el contenido de `ejemplos/` encima del proyecto y ejecutad:

terminal — php-paso-a-paso/clase38 — Visual Studio Code

terminal

```
php artisan migrate
```

Esquema

```
app/Events/AlumnoMatriculado.php              ← NUEVO: el evento
app/Listeners/                                ← NUEVOS: EnviarConfirmacionMatricula y
AvisarSiCursoCompleto
app/Mail/MatriculaConfirmada.php              ← NUEVO: el correo de confirmación
resources/views/mail/matricula-confirmada.blade.php ← NUEVA: su contenido
app/Notifications/                            ← NUEVAS: CursoCompleto y
RecordatorioInicioCurso
app/Jobs/EnviarRecordatoriosCurso.php         ← NUEVO: una tarea para la cola
app/Exceptions/SinPlazasException.php         ← NUEVA: una excepción propia
app/Http/Controllers/MatriculaController.php  ← NUEVO: matricular desde la ficha del curso
database/migrations/                          ← NUEVA: la tabla notifications
config/logging.php                            ← un canal de log propio
routes/console.php                            ← academia:recordatorios y las tareas
programadas
Alumno, Curso, CursoController, PaginaController, el panel, la ficha y las rutas
```



💡 La matrícula desde la ficha del curso era el ejercicio 02 de la clase 33. A partir de hoy forma parte del proyecto, y solo pueden usarla quienes pueden editar el curso.

En el `.env` ya tenemos desde la clase 28 tres líneas que hoy cobran sentido:

ejemplo.php — php-paso-a-paso/clase38 — Visual Studio Code

ejemplo.php

```
QUEUE_CONNECTION=database
CACHE_STORE=database
MAIL_MAILER=log
```



CLASE 38 · PUNTO 1

📘 Eventos y listeners

Un **evento** es un aviso de que **ha pasado algo** («se ha matriculado un alumno»). Los **listeners** («oyentes») son las clases que **reaccionan** a ese aviso. Quien lanza el evento no sabe cuántos oyentes hay ni qué hacen [1].

Esquema

```
                                  ┌──► EnviarConfirmacionMatricula   (correo al alumno)
MatriculaController               │
  AlumnoMatriculado::dispatch() ──┤
                                  │
                                  └──► AvisarSiCursoCompleto         (aviso a la dirección)
```

¿La ventaja? Mañana queremos además apuntar la matrícula en un programa de contabilidad. Creamos un tercer oyente y **no tocamos** el controlador.

terminal — php-paso-a-paso/clase38 — Visual Studio Code

terminal

```
php artisan make:event AlumnoMatriculado
php artisan make:listener EnviarConfirmacionMatricula --event=AlumnoMatriculado
```

El evento solo **lleva los datos** (`AlumnoMatriculado.php`):

ejemplo.php — php-paso-a-paso/clase38 — Visual Studio Code

ejemplo.php

```php
// Un evento: "ha pasado algo". Solo lleva los datos; no hace nada por sí mismo
class AlumnoMatriculado
{
    use Dispatchable, SerializesModels;

    public function __construct(
        public Alumno $alumno,
        public Curso $curso,
    ) {
    }
}
```

El controlador lo lanza (`MatriculaController.php`):

ejemplo.php — php-paso-a-paso/clase38 — Visual Studio Code

ejemplo.php

```php
$alumno = Alumno::findOrFail($datos['alumno_id']);
$curso->alumnos()->attach($alumno);

// Avisamos de que ha pasado algo. Quién reacciona y cómo ya no es cosa del controlador
AlumnoMatriculado::dispatch($alumno, $curso);
```

Y cada oyente recibe el evento en su método `handle()` (`AvisarSiCursoCompleto.php`):

ejemplo.php — php-paso-a-paso/clase38 — Visual Studio Code

ejemplo.php

```php
// Otro oyente del MISMO evento: cada uno hace una sola cosa
class AvisarSiCursoCompleto
{
    public function handle(AlumnoMatriculado $evento): void
    {
        $curso = $evento->curso;

        if ($curso->alumnos()->count() < $curso->plazas) {
            return;
        }

        $administradores = User::where('rol', Rol::Admin)->get();

        Notification::send($administradores, new CursoCompleto($curso));
    }
}
```

¿Cómo sabe Laravel qué oyentes tiene cada evento? **Mirando el tipo** del parámetro de `handle()` en las clases de `app/Listeners` . Comprobadlo:

terminal — php-paso-a-paso/clase38 — Visual Studio Code

terminal

```
php artisan event:list
```

Esquema

```
App\Events\AlumnoMatriculado
⇂ App\Listeners\AvisarSiCursoCompleto@handle
⇂ App\Listeners\EnviarConfirmacionMatricula@handle (ShouldQueue)
```



💡 Los modelos de Eloquent también lanzan eventos: `creating`, `created`, `saved`, `deleted` … Los usaremos en el punto 5 para vaciar la caché.



CLASE 38 · PUNTO 2

👍 Correo electrónico

Cada tipo de correo es una clase ***Mailable*** con su asunto y su vista [2]:

terminal — php-paso-a-paso/clase38 — Visual Studio Code

terminal

```
php artisan make:mail MatriculaConfirmada --markdown=mail.matricula-confirmada
```

`MatriculaConfirmada.php`:

ejemplo.php — php-paso-a-paso/clase38 — Visual Studio Code

ejemplo.php

```php
// Un correo: el asunto, la vista y los datos que necesita
class MatriculaConfirmada extends Mailable
{
    use Queueable, SerializesModels;

    // Las propiedades públicas llegan solas a la vista del correo
    public function __construct(
        public Alumno $alumno,
        public Curso $curso,
    ) {
    }

    public function envelope(): Envelope
    {
        return new Envelope(
            subject: "Matrícula confirmada: {$this->curso->titulo}",
        );
    }

    public function content(): Content
    {
        return new Content(
            markdown: 'mail.matricula-confirmada',
        );
    }
}
```

La vista se escribe en **Markdown** con componentes de correo, y Laravel la convierte en un correo HTML con buen aspecto en cualquier programa de correo (`matricula-confirmada.blade.php`):

plantilla.php — php-paso-a-paso/clase38 — Visual Studio Code

plantilla.php

```
<x-mail::message>
# ¡Hola, {{ $alumno->nombre }}! 👋

Tu matrícula en **{{ $curso->titulo }}** está confirmada.

<x-mail::table>
| Curso | Horas | Empieza |
|:------|:-----:|:-------:|
| {{ $curso->titulo }} | {{ $curso->horas }} | {{ $curso->fecha_inicio?->format('d/m/Y') ??
'por determinar' }} |
</x-mail::table>

<x-mail::button :url="route('cursos.show', $curso)">
Ver el curso
</x-mail::button>

¡Nos vemos en clase!<br>
{{ config('app.name') }}
</x-mail::message>
```

Y se envía con la fachada `Mail` (`EnviarConfirmacionMatricula.php`):

ejemplo.php — php-paso-a-paso/clase38 — Visual Studio Code

ejemplo.php

```
Mail::to($evento->alumno->email)
    ->send(new MatriculaConfirmada($evento->alumno, $evento->curso));
```

¿Adónde van los correos mientras desarrollamos?

Con `MAIL_MAILER=log`, los correos no salen: se escriben enteros en `storage/logs/laravel.log`. Perfecto para no enviar correos a direcciones inventadas por Faker. 😅 Otras opciones:

`MAIL_MAILER`	Qué hace
`log`	Escribe el correo en el log
`smtp`	Lo envía por un servidor SMTP. En local, con **Mailpit** (un programa gratuito que hace de servidor de correo de pruebas) lo veis en una bandeja de entrada en `http://localhost:8025`
`array`	Lo guarda en memoria (lo usan las pruebas)
`ses`, `postmark`, `resend` …	Servicios profesionales de envío para producción



💡 Si queréis ver cómo queda un correo sin enviarlo, una ruta que **devuelva** el *Mailable* lo pinta en el navegador: `return new MatriculaConfirmada($alumno, $curso);` .



CLASE 38 · PUNTO 3

😮 Colas y jobs

Enviar un correo puede tardar uno o dos segundos (conectar con el servidor, enviar…). ¿Tiene sentido que el usuario espere mirando la pantalla? No. Lo que no hace falta para responder se deja **para después**, en una **cola** [3].

Esquema

```
Petición  ──►  Controlador  ──►  guarda el trabajo en la cola  ──►  Respuesta inmediata
                                          │
                                          ▼   (en otra terminal, sin prisa)
                                 php artisan queue:work  ──►  envía el correo
```

Con `QUEUE_CONNECTION=database`, la cola es la tabla `jobs` (la creó la primera migración). Para que un oyente vaya a la cola, basta con implementar `ShouldQueue`:

ejemplo.php — php-paso-a-paso/clase38 — Visual Studio Code

ejemplo.php

```php
// ShouldQueue: no se ejecuta durante la petición, sino después, en la cola
class EnviarConfirmacionMatricula implements ShouldQueue
```

Y en otra terminal dejamos trabajando al ***worker*** («trabajador»):

terminal — php-paso-a-paso/clase38 — Visual Studio Code

terminal

```
php artisan queue:work
```

Esquema

```
2026-11-07 22:07:27 App\Notifications\CursoCompleto ........ 6.35ms DONE
2026-11-07 22:07:27 App\Listeners\EnviarConfirmacionMatricula ... 16.79ms DONE
```

Jobs: nuestras propias tareas

Un ***job*** es una tarea que se mete en la cola. `EnviarRecordatoriosCurso.php` avisa a todos los alumnos de un curso:

ejemplo.php — php-paso-a-paso/clase38 — Visual Studio Code

ejemplo.php

```php
// Un job: una tarea que se guarda en la cola y la ejecuta el "worker" cuando puede
class EnviarRecordatoriosCurso implements ShouldQueue
{
    use Queueable;

    // Si falla, se reintenta hasta 3 veces
    public int $tries = 3;

    public function __construct(
        public Curso $curso,
    ) {
    }

    public function handle(): void
    {
        Notification::send($this->curso->alumnos, new RecordatorioInicioCurso($this
->curso));

        Log::channel('academia')->info("Recordatorios enviados: {$this->curso->titulo}", [
            'alumnos' => $this->curso->alumnos->count(),
        ]);
    }
}
```

ejemplo.php — php-paso-a-paso/clase38 — Visual Studio Code

ejemplo.php

```php
EnviarRecordatoriosCurso::dispatch($curso);                      // a la cola
EnviarRecordatoriosCurso::dispatch($curso)->delay(now()->addHour()); // dentro de una hora
```

Comando	Qué hace
`php artisan queue:work`	Procesa la cola sin parar (hasta que pulséis Ctrl+C)
`php artisan queue:work --stop-when-empty`	Procesa lo pendiente y termina
`php artisan queue:failed`	Lista los trabajos que fallaron todos sus intentos
`php artisan queue:retry all`	Los vuelve a intentar



⚠️ El *worker* carga el código **al arrancar**. Si cambiáis un *listener* o un *job*, paradlo y volvedlo a lanzar, o seguirá ejecutando la versión antigua. Es el despiste número uno con las colas.



CLASE 38 · PUNTO 4

🚀 Notificaciones

Una ***notificación*** es un aviso que puede llegar por **varios canales** a la vez: correo, base de datos (para mostrarlo en la web), SMS, Slack... [4]. `CursoCompleto.php`:

ejemplo.php — php-paso-a-paso/clase38 — Visual Studio Code

ejemplo.php

```php
// Una notificación: un mismo aviso que puede llegar por varios canales
class CursoCompleto extends Notification implements ShouldQueue
{
    use Queueable;

    public function __construct(
        public Curso $curso,
    ) {
    }

    // Por qué canales llega: correo y la tabla notifications (para el panel)
    public function via(object $notifiable): array
    {
        return ['mail', 'database'];
    }

    public function toMail(object $notifiable): MailMessage
    {
        return (new MailMessage())
            ->subject("Curso completo: {$this->curso->titulo}")
            ->greeting("Hola, {$notifiable->name}")
            ->line("El curso «{$this->curso->titulo}» ha cubierto sus {$this->curso->plazas}
plazas.")
            ->action('Ver el curso', route('cursos.show', $this->curso))
            ->line('Quizá sea el momento de abrir otro grupo. 😉');
    }

    // Lo que se guarda en la columna data de la tabla notifications
    public function toArray(object $notifiable): array
    {
        return [
            'curso_id' => $this->curso->id,
            'mensaje' => "El curso «{$this->curso->titulo}» está completo.",
        ];
    }
}
```

Fijaos en que `toMail()` no necesita vista: `MailMessage` monta un correo con saludo, líneas y botón.

Se envían de dos formas:

ejemplo.php — php-paso-a-paso/clase38 — Visual Studio Code

ejemplo.php

```php
$usuario->notify(new CursoCompleto($curso));                    // a uno
Notification::send($administradores, new CursoCompleto($curso)); // a varios
```

Para poder recibir notificaciones, el modelo necesita el *trait* `Notifiable`. `User` ya lo trae; a `Alumno` se lo hemos añadido para los recordatorios. Por defecto, el canal `mail` usa su columna `email`.

Las notificaciones en el panel

El canal `database` guarda la notificación en la tabla `notifications` (la crea la migración de hoy, generada con `php artisan make:notifications-table`). En el panel las mostramos:

ejemplo.php — php-paso-a-paso/clase38 — Visual Studio Code

ejemplo.php

```php
@php
    $notificaciones = auth()->user()->unreadNotifications;
@endphp

<h2>🔔 Notificaciones ({{ $notificaciones->count() }})</h2>
@forelse ($notificaciones as $notificacion)
    <x-alerta>
        {{ $notificacion->data['mensaje'] }}
        <small>({{ $notificacion->created_at->diffForHumans() }})</small>
    </x-alerta>
@empty
    <p>No tienes notificaciones pendientes.</p>
@endforelse
```

`diffForHumans()` es de Carbon y escribe «hace 14 segundos» en el idioma de la aplicación. Y para marcarlas como leídas: `$request->user()->unreadNotifications->markAsRead()`.

	Mailable	Notificación
Para...	Un correo concreto con su propio diseño	Un aviso que puede ir por varios canales
Destinatario	Una dirección: `Mail::to('...')`	Un modelo `Notifiable`: `$usuario->notify(...)`
Ejemplo	La confirmación de matrícula	«El curso está completo» (correo + panel)



La portada hace varias consultas en **cada** visita, y los cursos cambian muy poco. ¿Por qué no guardar el resultado unos minutos? Eso es la **caché** [5]. `PaginaController.php`:

ejemplo.php — php-paso-a-paso/clase38 — Visual Studio Code

ejemplo.php

```php
// Las consultas de la portada se guardan 10 minutos en la caché
$datos = Cache::remember('portada', now()->addMinutes(10), function () {
    return [
        'totalCursos' => Curso::publicados()->count(),
        // Los tres que empiezan antes
        'destacados' => Curso::publicados()
            ->with('categoria')
            ->withCount('alumnos')
            ->orderBy('fecha_inicio')
            ->take(3)
            ->get(),
    ];
});

return view('paginas.inicio', $datos);
```

`Cache::remember()` hace esto: «¿tengo `portada` guardado? Lo devuelvo. ¿No? Ejecuto la función, guardo el resultado 10 minutos y lo devuelvo».

El problema de la caché: cuándo vaciarla

Si alguien crea un curso, la portada seguiría enseñando los datos viejos durante 10 minutos. Lo arreglamos con los **eventos del modelo**: cada vez que un curso se guarda o se borra, olvidamos la portada (`Curso.php`):

ejemplo.php — php-paso-a-paso/clase38 — Visual Studio Code

ejemplo.php

```php
// Eventos del modelo: cada vez que un curso se guarda o se borra, la portada en caché
caduca
protected static function booted(): void
{
    static::saved(fn () => Cache::forget('portada'));
    static::deleted(fn () => Cache::forget('portada'));
}
```

Método	Qué hace
`Cache::remember('clave', $tiempo, fn)`	Devuelve lo guardado o lo calcula y lo guarda
`Cache::rememberForever('clave', fn)`	Igual, sin caducidad
`Cache::get('clave', $porDefecto)` / `Cache::put('clave', $valor, $tiempo)`	Leer / guardar
`Cache::forget('clave')`	Olvidar una clave
`php artisan cache:clear`	Vaciar toda la caché

Con `CACHE_STORE=database`, la caché es la tabla `cache`. En producción se suele usar **Redis**, mucho más rápido, y no hay que cambiar ni una línea de código (¿recordáis los contratos de la clase 35?).



⚠️ Hay un dicho famoso en informática: «solo hay dos cosas difíciles: invalidar la caché y ponerle nombre a las cosas». Usad la caché solo donde haga falta y pensad siempre **cuándo** debe caducar.



CLASE 38 · PUNTO 6

🤔 Tareas programadas

Los recordatorios hay que enviarlos **todos los días** a la misma hora. Para eso existe el ***scheduler*** («planificador») de Laravel. Primero, el comando que hace el trabajo (`routes/console.php`):

ejemplo.php — php-paso-a-paso/clase38 — Visual Studio Code

ejemplo.php

```php
// php artisan academia:recordatorios: un job a la cola por cada curso que empieza mañana
Artisan::command('academia:recordatorios', function () {
    $cursos = Curso::publicados()->whereDate('fecha_inicio', today()->addDay())->get();

    foreach ($cursos as $curso) {
        EnviarRecordatoriosCurso::dispatch($curso);
    }

    $this->info("Recordatorios en cola: {$cursos->count()} cursos.");
})->purpose('Avisa a los alumnos de los cursos que empiezan mañana');
```

Y después, **cuándo** se ejecuta, en el mismo fichero [6]:

ejemplo.php — php-paso-a-paso/clase38 — Visual Studio Code

ejemplo.php

```php
// TAREAS PROGRAMADAS: el planificador las ejecuta cuando toca (php artisan schedule:work)
Schedule::command('academia:recordatorios')->dailyAt('09:00');
Schedule::command('sanctum:prune-expired --hours=24')->daily();
```

La segunda tarea borra cada día los tokens caducados de la API (clase 36): limpieza automática.

Método	Cuándo
`->everyMinute()`, `->everyFiveMinutes()`	Cada minuto / cada cinco minutos
`->hourly()`, `->daily()`, `->dailyAt('09:00')`	Cada hora / cada día a medianoche / cada día a las 9
`->weekly()`, `->mondays()->at('08:00')`	Cada semana / los lunes a las 8
`->monthly()`	El primer día de cada mes

terminal — php-paso-a-paso/clase38 — Visual Studio Code

terminal

```
php artisan schedule:list    # qué tareas hay y cuándo toca la siguiente
php artisan schedule:work    # en desarrollo: lo deja vigilando y ejecuta cada tarea a su hora
```

En un servidor de verdad no se deja una terminal abierta: se configura **una sola línea** en el `cron` del sistema que ejecuta `php artisan schedule:run` cada minuto, y Laravel decide qué toca. Lo veremos en la clase 39.



Laravel apunta los errores en `storage/logs/laravel.log`, y nosotros podemos apuntar lo que queramos con la fachada `Log` [7]:

ejemplo.php — php-paso-a-paso/clase38 — Visual Studio Code

ejemplo.php

```php
Log::info('Se ha matriculado un alumno', ['alumno_id' => $alumno->id]);
Log::warning('Quedan pocas plazas');
Log::error('No se pudo conectar con la pasarela de pago');
```

Hay ocho niveles, de menos a más grave: `debug`, `info`, `notice`, `warning`, `error`, `critical`, `alert` y `emergency`. El segundo parámetro es el **contexto**: un array con datos que se guardan junto al mensaje.

Un canal propio

Para no mezclar lo de la academia con los errores de Laravel, añadimos un **canal** en `config/logging.php`:

ejemplo.php — php-paso-a-paso/clase38 — Visual Studio Code

ejemplo.php

```php
// Nuestro canal: un fichero por día en storage/logs/academia-AAAA-MM-DD.log
'academia' => [
    'driver' => 'daily',
    'path' => storage_path('logs/academia.log'),
    'level' => 'info',
    'days' => 30,
],
```

ejemplo.php — php-paso-a-paso/clase38 — Visual Studio Code

ejemplo.php

```php
Log::channel('academia')->info("Recordatorios enviados: {$this->curso->titulo}", [...]);
```

Esquema

```
[2026-11-07 22:07:41] local.INFO: Recordatorios enviados: PHP desde cero {"alumnos":6}
```

El *driver* `daily` crea un fichero por día y borra los de más de 30 días: así el log no crece hasta llenar el disco.



💡 `php artisan pail` muestra los logs en directo en la terminal, a medida que se escriben. Muy útil para ver qué está pasando mientras probáis la web.



CLASE 38 · PUNTO 8

⚠️ Excepciones propias

En la clase 21 creamos nuestras propias excepciones. En Laravel, una excepción puede decidir además **cómo se muestra** y **qué se apunta en el log** [8]. `SinPlazasException.php`:

ejemplo.php — php-paso-a-paso/clase38 — Visual Studio Code

ejemplo.php

```php
// Una excepción propia que sabe cómo mostrarse (render) y cómo apuntarse en el log
(context)
class SinPlazasException extends Exception
{
    public function __construct(
        public Curso $curso,
    ) {
        parent::__construct("No quedan plazas en el curso «{$curso->titulo}».");
    }

    // Datos extra que se guardan en el log junto al error
    public function context(): array
    {
        return ['curso_id' => $this->curso->id, 'plazas' => $this->curso->plazas];
    }

    // Lo que ve el usuario: vuelve atrás con el mensaje de error
    public function render(): RedirectResponse
    {
        return back()->with('error', $this->getMessage());
    }
}
```

Y el controlador solo tiene que lanzarla:

ejemplo.php — php-paso-a-paso/clase38 — Visual Studio Code

ejemplo.php

```php
if ($curso->alumnos()->count() >= $curso->plazas) {
    throw new SinPlazasException($curso);
}
```

Laravel la **apunta** en el log (con el contexto) y la **muestra** con su `render()`. El controlador queda limpio, y si mañana se matricula desde otro sitio, el comportamiento es el mismo.



💡 Si un error es «normal» y no queréis llenar el log con él, se le puede decir a Laravel que no lo apunte en `bootstrap/app.php`: `$exceptions->dontReport(SinPlazasException::class);`.



CLASE 38 · PUNTO 9

🚀 El recorrido completo de una matrícula

Entrad como Marta (editora), id a la ficha de un curso y matriculad a un alumno. Esto es lo que pasa:

Esquema

```
1. MatriculaController valida, comprueba plazas y hace attach()     (en la petición)
2. AlumnoMatriculado::dispatch()                                     (en la petición)
      ├─ AvisarSiCursoCompleto: ¿se ha llenado? → notificación       (en la petición)
      │     └─ CursoCompleto implements ShouldQueue → a la cola
      └─ EnviarConfirmacionMatricula implements ShouldQueue → a la cola
3. Respuesta: «Alumno matriculado» ✅                                (¡ya! sin esperar a los correos)

 ... en la otra terminal, php artisan queue:work:
4. Envía el correo de confirmación al alumno
5. Envía el aviso por correo al administrador y lo guarda en notifications
6. Ramón entra en el panel: 🔔 «El curso PHP desde cero está completo»
```

Para probarlo rápido, dejad un curso con una plaza libre:

terminal — php-paso-a-paso/clase38 — Visual Studio Code

terminal

```
php artisan tinker
> $curso = App\Models\Curso::withCount('alumnos')->find(1); $curso->update(['plazas' => $curso->alumnos_count + 1]);
```



CLASE 38 · PUNTO 10

🚀 Ejercicios

Las soluciones se copian dentro del proyecto, respetando sus carpetas.

- ☐ **01 · El contacto, por correo.** Cread el *Mailable* `MensajeContacto` (que vaya siempre a la cola: `implements ShouldQueue`) con una vista Markdown, y una nueva implementación del contrato de la clase 35, `NotificadorCorreo`. Cambiad **una línea** del *provider* para que el formulario de contacto envíe correos. → `MensajeContacto.php`, `mensaje-contacto.blade.php`, `NotificadorCorreo.php` y `AcademiaServiceProvider.php`
- ☐ **02 · Probar la matrícula.** Escribid pruebas (clase 37) que comprueben que al matricular se lanza el evento (`Event::fake()` y `Event::assertDispatched()`), que el alumno recibe el correo (`Mail::fake()` y `Mail::assertSent()`), que si el curso se llena se notifica al administrador (`Notification::fake()` y `Notification::assertSentTo()`) y que sin plazas no se matricula. → `MatriculaTest.php`

- ☐ **03 · Las categorías del menú, en caché.** El *view composer* de la clase 35 consulta las categorías en **cada** página. Guardadlas para siempre con `Cache::rememberForever()` y olvidadlas cuando una categoría se guarde o se borre. → `AcademiaServiceProvider.php` y `Categoria.php`



🏆 **Reto:** cread el comando `academia:informe-semanal`, que envíe a los administradores una notificación con los cursos que empiezan esa semana y cuántos alumnos tiene cada uno, y programadlo para los lunes a las 8:00.



CLASE 38 · PUNTO 11

👋 Lo que tenéis que llevaros de hoy

Idea	En una frase
Evento	«Ha pasado algo»: `AlumnoMatriculado::dispatch(...)`
Listener	Reacciona a un evento; uno por tarea
Mailable	Un correo con su asunto y su vista Markdown; `Mail::to()->send()`
`MAIL_MAILER=log`	En desarrollo, los correos se escriben en el log (o en Mailpit)
Cola	Lo que no hace falta para responder, para después: `ShouldQueue`
`queue:work`	El trabajador que vacía la cola (reiniciadlo al cambiar el código)
Job	Una tarea propia para la cola: `dispatch()`, `delay()`, `$tries`
Notificación	Un aviso por varios canales: `mail`, `database`…
Caché	`Cache::remember()`; y pensar siempre cuándo debe caducar
Scheduler	`Schedule::command()->dailyAt()`; en el servidor, una línea de `cron`
Logs	`Log::info()` con contexto; canales propios con `daily`
Excepción propia	Con `render()` y `context()`: el controlador solo la lanza

📚 Fuentes y documentación oficial

1. Laravel: Events
2. Laravel: Mail
3. Laravel: Queues
4. Laravel: Notifications
5. Laravel: Cache
6. Laravel: Task Scheduling
7. Laravel: Logging
8. Laravel: Error Handling